TrazaCao (E07 Instalación respaldo y mantenimiento)

1. Instalación y ejecución reproducible

TrazaCao utiliza Python, FastAPI, SQLite y un frontend web. No requiere contratar servicios de nube ni instalar un servidor de base de datos independiente.

Procedimiento de instalación:

1. Clonar el repositorio y crear el entorno de Python:

git clone https://github.com/TheKelex/TrazaCao.git

cd TrazaCao

python -m venv .venv

2. Activar el entorno virtual e instalar las dependencias:

python -m pip install -r src/backend/requirements.txt

3. Preparar los datos sintéticos iniciales únicamente en una base de demostración limpia:

cd src/backend

python seed.py

4. Iniciar el backend desde src/backend:

uvicorn main:app --port 8000

La API queda disponible en http://localhost:8000

5. Iniciar el frontend desde src/frontend, instalando las dependencias declaradas en su archivo de configuración. Los comandos definitivos de instalación y ejecución deben quedar documentados en el README y verificarse antes de la entrega.

Pruebas: desde la raíz del repositorio, ejecutar pytest tests/ -v. El README debe documentar los usuarios de demostración, los datos sintéticos, las dependencias, las versiones y los comandos comprobados para iniciar la aplicación.

Verificación de instalación limpia: registrar el resultado de una ejecución realizada en un entorno preparado desde cero, indicando sistema operativo, versiones y posibles errores encontrados.

2. Respaldo, restauración y continuidad operativa

La base de datos SQLite se crea por defecto como cacao_dev.db en la raíz del proyecto. La variable DATABASE_PATH permite configurar una ubicación diferente.

Crear respaldo: desde la raíz del repositorio, ejecutar:

python scripts/backup.py

El script genera una copia con fecha en datos/respaldos/, una copia denominada datos/cacao_backup_latest.db y un archivo de checksum SHA-256.

Restaurar respaldo: detener el backend y ejecutar:

python scripts/restore.py

Este comando utiliza la copia más reciente. También se puede indicar como argumento la ruta de otro archivo de respaldo. Si se ha configurado DATABASE_PATH, debe mantenerse la misma configuración durante la restauración.

3. Servicios externos, costos y fallos conocidos

Servicios externos y costos: el sistema no requiere servicios de nube ni APIs externas durante su operación local prevista.

Operación sin conexión externa: una vez instaladas las dependencias, la aplicación puede operar localmente con SQLite sin depender de un servicio remoto.

Fallos conocidos y mitigaciones:

- **Escrituras concurrentes:** SQLite puede experimentar contención cuando varias operaciones intentan escribir simultáneamente.
- **Restauración durante la operación:** se debe detener el backend antes de sustituir la base de datos, realizar una copia preventiva y comprobar los datos recuperados.
- **Reinicio de datos de demostración:** ejecutar seed.py sobre registros existentes puede eliminar información. Debe utilizarse únicamente en la preparación inicial de una base de prueba.

4. Plan de mantenimiento a 30 días

Prioridad y mejora	Responsable	Esfuerzo	Criterio de aceptación
1. Fortalecer la recuperación de datos: verificar checksum, identificadores y cantidades después de restaurar.	Backend e infraestructura	8 h	Restauración de prueba con integridad correcta y comparación satisfactoria de registros y cantidades.
2. Mejorar usabilidad y accesibilidad: aplicar los ajustes detectados en E06 y verificar la experiencia móvil.	Frontend y experiencia de usuario	12 h	Registrar un lote y consultar su trazabilidad sin instrucciones paso a paso; documentar resultados y evidencias.
3. Ampliar las pruebas automatizadas: cubrir permisos, consumo excesivo, duplicación de operaciones y cambios de estado.	Backend y pruebas	12 h	Casos automatizados ejecutados correctamente, resultados registrados y regresiones documentadas.